ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»
Факультет информатики, математики и компьютерных наук

Ковалева Варвара Александровна

ПОСТРОЕНИЕ ЛОГИСТИЧЕСКОЙ ТРАНСПОРТНОЙ СЕТИ С НЕЛИНЕЙНОЙ СТОИМОСТЬЮ РЁБЕР И УЧЁТОМ ПЕРЕГРУЗА

Выпускная квалификационная работа - МАГИСТЕРСКАЯ РАБОТА
по направлению подготовки Прикладная математика и информатика
образовательная программа
«Интеллектуальный анализ данных»

Рецензент
к-т физ.-мат. наук, доцент
М.В. Бацын

Руководитель
к-т технических наук, доцент
А.А. Пономаренко

Нижний Новгород, 2026

Оглавление

Введение 4

Глава 1. Описание задачи 7

1.1. Содержательная постановка задачи 7

1.2. Математическая постановка задачи 7

1.3. Классификация задачи и ее релаксация 10

Глава 2. Предлагаемый подход решения задачи 12

2.1. General Variable Neighborhood Search подход 12

2.2. Предобработка данных и инициализация начального решения 15

2.3. Окрестности GVNS 16

2.3.1. Окрестность кратчайших путей 17

2.3.2. Окрестность частично загруженных ребер 18

2.3.3. Окрестность обхода перегруженных вершин 20

2.4. Итоговый вид предлагаемого алгоритма 21

Глава 3. Проведение эксперимента 22

3.1. Постановка задачи для эксперимента 22

3.2. Генерация синтетических данных для проведения эксперимента 23

3.2.1. Формирование графов на основе реальных данных 23

3.2.2. Анализ характеристик сформированных графов 25

3.2.3. Алгоритм генерации данных 28

3.2.4. Составление набора тестовых данных 36

3.3. Критерии оценки 36

3.4. Аппаратные характеристики 37

3.5. Результаты эксперимента 37

Заключение 43

Список использованной литературы 44

Приложение 1. Характеристики сгенерированных данных 46

Приложение 2. Результаты эксперимента 48

Введение

Проблема организации товарных потоков между распределительными центрами была и остается одной из ключевых задач логистических сетей. Стремительный рост сложности и масштаба современных складских структур порождает необходимость в разработке все более эффективных алгоритмов построения оптимальной системы перевозок товаров. При этом реальные сети характеризуются нелинейной стоимостью перевозок (из-за использования транспортных средств фиксированной грузовместимости) и ограниченной пропускной способностью складов, что существенно усложняет постановку и решение задачи.

Традиционные точные методы решения многопродуктовых потоковых задач (Multi-Commodity Flow Problem) требуют больших вычислительных затрат, в связи с их NP-трудностью. Поэтому особую значимость приобретают различные приближенные алгоритмы, позволяющие получить качественное решение, близкое к оптимальному, за приемлемое время.

Одним из известных походов является метод генерации столбцов (Column Generation), который позволяет решать задачи путём динамического построения множества перспективных путей [1]. Данный метод особенно эффективен в путевой постановке задачи и часто используется в сочетании с Branch-and-Price.

В практике также распространены различные эвристические и метаэвристические алгоритмы, такие как жадные алгоритмы, генетические алгоритмы, tabu search, simulated annealing, а также методы на основе локального поиска [2]. Кроме этого, применяются агрегированные методы, например, группирующие похожие товары или объединяющие близкие вершины.

Особое внимание заслуживает такая метаэвристика, как Variable Neighborhood Search (VNS) и его вариации. Она находит применение в абсолютно разных оптимизационных задачах. В частности, в работе Masri H. от 2015 года успешно применили multi-start VNS для решения single-path multicommodity flow problem [3]. Также высокую эффективность

продемонстрировал General VNS при решении capacitated vehicle routing problem и других логистических задач с потоковыми ограничениями [4].

Целью данной выпускной квалификационной работы является разработка и исследование метаэвристического алгоритма на основе General Variable Neighborhood Search для решения задачи распределения потоков движения товаров в логистической сети с учетом нелинейной стоимости ребер и ограниченной пропускной способности складов.

Для достижения данной цели были поставлены следующие задачи:

- исследовать проблематику поставленной задачи, построить математическую модель и формализовать критерий оптимальности;
- изучить структуру GVNS подхода и особенности определения окрестностей;
- разработать набор специализированных окрестностей для алгоритма GVNS, учитывающих специфику задачи, и реализовать предлагаемый подход;
- проанализировать структуру и ключевые характеристики реальных логистических сетей и на их основе разработать комплексный подход к генерации правдоподобных синтетических данных;
- провести вычислительные эксперименты на синтетических и реальных данных и выполнить сравнительный анализ эффективности предложенного подхода.

Предметом данного исследования является изучение и применение метаэвристических методов решения задачи многопродуктового потока минимальной стоимости в логистической сети с ограничениями на максимальной перегруз складов и нелинейной стоимостью ребер. Основным методом решения данной оптимизационной задачи выбран GVNS, который демонстрирует высокую эффективность при решении комбинаторных задач и гибкость в формировании окрестностей.

Научная новизна работы состоит в предложении специализированных окрестностей GVNS, адаптированных к путевой постановке данной задачи и учитывающих ее ограничения, а также в создании комплексного подхода генерации логистических сетей. Полученные результаты могут быть использованы как для развития предложенного метода решения такого рода задач, так и в практических целях для планирования перевозок в логистических системах.

Данная магистерская выпускная квалификационная работа состоит из 3 глав, введения, заключения, списка использованной литературы и приложений. Первая глава включает в себя содержательную и математическую поставки задачи в потоковой и путевой версии, а также ее релаксированную версию. Вторая глава представляет собой описание GVNS подхода и предлагаемых структур окрестностей. Третья глава содержит постановку эксперимента, описание метода генерации данных для поставленной задачи и анализ полученных результатов. В Приложении 1 представлены таблицы с перечислением характеристик сгенерированных для проведения эксперимента данных. В Приложении 2 приведены результаты вычислительных экспериментов в виде таблиц.

Глава 1. Описание задачи

1.1. Содержательная постановка задачи

Рассматривается задача оптимальной организации перевозок товара (груза) между складами, расположенных в транспортной сети. Необходимо перевезти заданные объемы товара из пунктов отправления в пункты назначения, минимизируя суммарные затраты на движение транспортных средств по дорогам и расходы на перегрузку (выгрузка из одного транспортного средства, хранение, и погрузку в другой) в промежуточных (транзитных) складах.

Каждый склад обладает ограниченной пропускной способностью (максимальным объемом перегруза) и стоимостью выполнения перегрузочных операций для единицы товара. Перевозка груза осуществляется транспортными средствами фиксированного объема. Для каждого участка дороги задана стоимость проезда одного транспортного средства по ней. Количество транспортных средств на каждом участке должно быть достаточным для перевозки суммарного объема товара, проходящего по этому ребру.

Задача относится к классу многопродуктовых потоковых задач (Multi Commodity Flow Problem) с дополнительными ограничениями на пропускную способность вершин и с нелинейной стоимостью ребер за счет использования транспортных средств. Данная задача может быть сформулирована в двух эквивалентных постановках: потоковой (arc-based) и путевой (path-based).

1.2. Математическая постановка задачи

Исходная информация

Задана сеть транспортных дорог в виде ориентированного графа $G = (V, E)$, где V – множество складов, E – множество дорог между ними.

Дано множество запросов на перевозку грузов K, в котором для каждого $k \in K$ известно:

- $s_k \in V$ – пункт отправления;

- $t_k \in V$ – пункт назначения;
- $d_k \in \mathbb{R}_+$ – объем груза, который необходимо перевезти из s_k в t_k.

Для каждого склада $v \in V$ известна следующая информация:

- $f_v \in \mathbb{R}_+$ – стоимость перегрузки единицы товара;
- $W_v \in \mathbb{R}_+$ – максимально допустимый объем перегруза.

Каждое ребро $e \in E$ обладает характеристикой:

- $c_e \in \mathbb{R}_+$ – затраты на проезд одного транспортного средства

Также задан объем грузового транспортного средства $C \in \mathbb{R}_+$, в котором перевозят товары.

Варьируемые параметры модели

Для каждого варианта постановки задачи введем следующие обозначения.

Потоковая постановка:

- $x_e^k \in \mathbb{R}_+$ – поток товара $k \in K$, проходящего по ребру $e \in E$;
- $y_e \in \mathbb{Z}_+$ – количество транспортных средств, проезжающих по ребру $e \in E$.

Общее количество переменных составляет $|K| * |E| + |E|$

Путевая постановка:

Пусть для каждого товара $k \in K$ известно $П_k$ – множество всех возможных путей из s_k в t_k.

- $x_p^k \in \mathbb{R}_+$ – поток товара k по пути $p \in П_k$;
- $y_e \in \mathbb{Z}_+$ – количество транспортных средств, проезжающих по ребру $e \in E$.

Общее количество переменных составляет $\sum_{k \in K} П_k + |E|$

Целевая функция

Необходимо распределить товары и транспортные средства по дорогам так, чтобы минимизировать суммарную стоимость перевозок (первое слагаемое) и перегруза в транзитных вершинах (второе слагаемое).

Потоковая постановка:

Обозначим через $\delta^-(v) \subseteq E$ и $\delta^+(v) \subseteq E$ входящие и исходящие ребра вершины $v \in V$ соответсвенно.

$$min\left(\sum_{e\in E} c_e * y_e + \sum_{k\in K} \sum_{\substack{v\in V,\\ v\neq s_k,\, v\neq t_k}} \sum_{e\in\delta^-(v)} f_v * x_e^k\right) \tag{1.1}$$

Путевая постановка:

$$min\left(\sum_{e\in E} c_e * y_e + \sum_{k\in K} \sum_{p\in \Pi_k} \sum_{\substack{v\in p\\ v\neq s_k,\, v\neq t_k}} f_v * x_p^k\right) \tag{1.2}$$

Ограничения модели

1. Для каждого товара необходимо удовлетворить заданный спрос:

Потоковая постановка:

$$\sum_{e\in\delta^+(v)} x_e^k - \sum_{e\in\delta^-(v)} x_e^k = \begin{cases} d_k, & v = s_k \\ -d_k, & v = t_k \\ 0, & \text{иначе} \end{cases}, \quad \forall k \in K, \forall v \in V \tag{2.1}$$

Путевая постановка:

$$\sum_{p\in\Pi_k} x_p^k = d_k, \quad \forall k \in K \tag{2.2}$$

2. По каждому ребру должно проезжать такое количество транспортных средств, чтобы в них можно было вместить весь суммарный объем товара, перевозимого по этому ребру:

Потоковая постановка:

$$\sum_{k\in K} x_e^k \leq C * y_e, \quad \forall e \in E \tag{3.1}$$

Путевая постановка:

$$\sum_{k\in K} \sum_{p\in\Pi_k : e\in p} x_p^k \leq C * y_e, \quad \forall e \in E \tag{3.2}$$

3. На каждый склад задано ограничение на максимальный суммарный объем перегружаемого товара:

Потоковая постановка:

$$\sum_{\substack{k\in K,\\ v\neq s_k,\, v\neq t_k}} \sum_{e\in\delta^-(v)} x_e^k \le W_v,\ \ \forall v \in V \tag{4.1}$$

Путевая постановка:

$$\sum_{\substack{k\in K,\\ v\neq s_k,\, v\neq t_k}} \sum_{p\in\Pi_k} \sum_{v\in p} x_p^k \le W_v,\ \ \forall v \in V \tag{4.2}$$

1.3. Классификация задачи и ее релаксация

Поскольку целевая функция и все ограничения модели являются линейными, а среди переменных присутствуют как целочисленные (количество транспортных средств на ребре), так и непрерывные (поток товара на ребре или пути), то данную задачу можно классифицировать как задачу смешенного целочисленного линейного программирования (Mixed Integer Linear Programming).

Данный тип задач относится к классу NP-трудных, и в общем случае для них не существует алгоритма, гарантирующего получение оптимального решения за полиномиальное время. В связи с этим на практике широко используются различные подходы, позволяющие получить качественное приближенное решение за приемлемое время.

Одним из наиболее распространенных методов получения оценки целевой функции таких задач является линейная релаксация (Linear Programming relaxation). Данный подход заключается в снятии ограничения на целочисленность переменных, в результате чего исходная MILP-задача сводится к задаче линейного программирования (LP), для которой существуют эффективные полиномиальные алгоритмы решения. Оптимальное значение релаксированной задачи дает нижнюю (в случае минимизации) или верхнюю (в случае максимизации) границу для исходной модели. Полученное решение часто используется для построения начальных границ в алгоритме ветвей и границ, для быстрой оценки качества эвристических решений, или в качестве

начального решения для других методов, если после округлении дробных значений оно остается допустимым.

Для получения релаксированной версии рассматриваемой в рамках данной работы задачи необходимо убрать ограничение на целочисленность переменной y_e – количества транспортных средств на ребре. В следствии этого ограничения (3.1) и (3.2) можно преобразовать в равенство и выразить y_e через переменные x_e^k и x_p^k соответственно:

Потоковая постановка:

$$y_e = \frac{1}{C}\sum_{k\in K} x_e^k, \ \forall e \in E \tag{5.1}$$

Путевая постановка:

$$y_e = \frac{1}{C}\sum_{k\in K}\sum_{p\in \Pi_k : e\in p} x_p^k, \ \forall e \in E \tag{5.2}$$

Таким образом при подстановке полученных выражений в целевую функцию получим релаксированную задачу, зависящую только от переменных x_e^k и x_p^k, то есть от потоков по ребрам или путям:

Потоковая постановка:

$$min\left(\sum_{k\in K}\sum_{e\in E} x_e^k * \left(\frac{c_e}{C} + \sum_{\substack{v\in V,\\ v\neq s_k, v\neq t_k,\\ e\in\delta^-(v)}} f_v\right)\right) \tag{6.1}$$

Путевая постановка:

$$min\left(\sum_{k\in K}\sum_{p\in \Pi_k} x_p^k * \left(\frac{1}{C}\sum_{e\in p} c_e + \sum_{\substack{v\in p\\ v\neq s_k, v\neq t_k}} f_v\right)\right) \tag{6.2}$$

Глава 2. Предлагаемый подход решения задачи

2.1. General Variable Neighborhood Search подход

В основу разработанного в рамках данной работы алгоритма лег Variable Neighborhood Search (VNS) подход [5], [6]. VNS – это метаэвристика для решения оптимизационной задачи, главной идеей которого является систематическая смена структуры окрестностей возможных решений для всестороннего изучения пространства поиска.

Пусть мы рассматриваем оптимизационную задачу $\min\{f(x)|x \in X\}$ и задан набор из k_{max} структур окрестностей $N_k(x)(k = 1, \dots, k_{max})$, которые определяют множество решений на основе x.

Классическая структура VNS метода состоит из трех этапов:

1. Shake: случайно изменяем текущее решение используя одну из структур окрестностей. Целью этапа является выход из локального оптимума и попадание в другую область пространства решений.

Алгоритм 1 – *Shake*-функция

Входные данные: решение $x \in X$, номер окрестности k.

Выходные данные: новое решение $x' \in N_k(x)$.

def Shake(x, k):

1 $x' \leftarrow$ rand_elem$(N_k(x))$ // случайное решение из k-ой окрестности

return x'

2. Local Search: на основе полученного в первом этапе решения ищем локальный оптимум в его окрестности. Для этого используются разные подходы, например, best-improvement или first-improvement.

3. Solution Move and Neighborhood Change: сравниваем найденный локальный оптимум и текущее лучшее решение и, если первое оказалось лучше, то продолжаем поиск, снова начиная с первой окрестности с новым текущим

решением, иначе пытаемся улучшить решение с помощью следующей окрестности.

Алгоритм 2 – *Neighborhood Change*

Входные данные: текущее решение $x \in X$, новое решение $x' \in X$, текущий номер окрестности k.

Выходные данные: обновленные текущие решение x и окрестность k.

def NeighborhoodChange(x, x', k):

1 **if** $f(x') < f(x)$ **then**

2 | $x \leftarrow x'$ // Смена текущего решения

3 | $k \leftarrow 1$ // Изначальная окрестность

4 **else**

5 | $k \leftarrow k + 1$ // Смена окрестности на следующую

return x, k

Одним из мощных методов локального поиска является Variable Neighborhood Descent (VND), который также использует идею перебора окрестностей детерминированным образом. Данный подход основан на том, что локальное решение, которое является оптимальным по отношению к нескольким структурам окрестностей, с большей вероятностью будет глобальным оптимумом, чем решение, найденное в рамках локального поиска в одной окрестности. Кроме того, важным аспектом является порядок рассматриваемых структур окрестностей, которые обычно располагаются так, что после фазы интенсивного улучшения решения (интенсификации) постепенно наступает фаза более широкого исследования пространства.

Алгоритм 3 – *Variable Neighborhood Descent*

Входные данные: текущее решение $x \in X$, количество окрестности k_{max}.

Выходные данные: новое решение x.

def VND(x, k_{max}):

1 $k \leftarrow 1$

2 **while** $k \leq k_{max}$ **do**

3 | $x' \leftarrow argmin_{y \in N_k(x)}(f_y)$ // Поиск лучшего решения из $N_k(x)$

4 | $x, k \leftarrow \text{NeighborhoodChange}(x, x', k)$ // Смена окрестности

5 **end while**

return x

В качестве метода локального поиска во втором этапе VNS можно использовать VND. Данный поход называется General Variable Neighborhood Search (GVNS), которые известен в литературе как гибкий инструмент, применимый к разного рода оптимизационным задачам и показывающий успешные результаты по сравнению с другими эвристиками (источники [7], [8]). Стоит заметить, что набор окрестностей и их количество, используемых для Shake-функции и VND, не обязательно должны совпадать, хотя чаще всего они все-таки одинаковы.

Алгоритм 4 – *General Variable Neighborhood Search*

Входные данные: начальное решение $x \in X$, количества окрестностей k_{max}, l_{max}, максимальное время выполнения t_{max}.

Выходные данные: решение x.

---

def $\text{GVNS}(x, k_{max}, l_{max}, t_{max})$:

1 $k \leftarrow 1$

2 **repeat**

3 | $x' \leftarrow \text{Shake}(x, k)$ // Случайное решение из окрестности

4 | $x'' \leftarrow \text{VND}(x', l_{max})$ // Локальный поиск

5 | $x, k \leftarrow \text{NeighborhoodChange}(x, x'', k)$ // Смена окрестности

6 | $t \leftarrow \text{CpuTime}()$

7 **until** $t > t_{max}$ **or** $k > k_{max}$

return x

2.2. Предобработка данных и инициализация начального решения

Транспортные сети чаще всего представляют собой разреженные ориентированные графы, в которых далеко не все вершины соединены прямыми дорогами. Также при планировании маршрута транспортного средства не обязательно во всех промежуточных пунктах необходимо выполнять перегрузку товара. Если при проезде через транзитный склад не будет происходить загрузка нового или выгрузка привезённого в конечный пункт товара, то и нет необходимости в нем останавливаться.

Также использование «сырого» исходного графа приводит к существенному сужению пространства допустимых решений (за счет ограничения на пропускную способность склада) и может сделать задачу неразрешимой (не будет возможности удовлетворить спрос на перевозку груза из-за отсутствия пути из пункта отправления в пункт назначения).

Для устранения вышеописанных проблем необходимо произвести предобработку графа. Она заключается в достраивании его до полного графа путем добавления (или обновления) ребра между складами v и u, стоимость проезда по которому соответствует стоимости кратчайшего пути из v в u. В результате получаем полный граф, где каждое ребро представляет собой наиболее экономически выгодный путь между своими конечными вершинами без остановок в промежуточных пунктах.

Кроме того, структура полученного графа гарантирует существование допустимого решения задачи, так как поток товара по ребру, соединяющего его пункты отправки и назначения, не затрагивает другие вершины и, следовательно, не нарушает ограничения максимального суммарного объема перегруза в узлах.

Из анализа целевой функции релаксированной задачи в путевой постановке (6.2) следует, что оптимальное решение стремиться пустить поток товаров по путям с наименьшей стоимостью и минимальным количеством транзитных вершин. В сформированном графе такой характеристикой обладают именно ребра, соединяющие пункты отправления и назначения товаров, так как

его стоимость соответствует стоимости кратчайшего пути и на нем не происходит перегрузки товара.

В связи с этим в качестве начального решения алгоритма предлагается использовать решение релаксированной LP-задачи, в котором весь объем каждого груза направляется по прямому ребру из источника в сток. Значения же переменных по количеству транспортных средств на ребрах устанавливается равным округленному вверх результату деления суммарного потока по ребру на объем транспортного средства:

$$y_e = \left\lceil \frac{1}{C} \sum_{k \in K} x_e^k \right\rceil, \ \forall e \in E \tag{7}$$

2.3. Окрестности GVNS

Так как значения переменных y_e напрямую вычисляются из переменных x_e^k и x_p^k (выражение (7)), то в рамках окрестностей более целесообразно варьировать именно либо поток по ребрам, либо по путям.

Предугадать допустимые значения потоков по отдельным ребрам может быть проблематично, поскольку один участок дороги может использоваться для перевозки множества различных товаров, а потоки по соседним ребрам сильно взаимосвязаны. В результате для обеспечения допустимого решения требуется учитывать всю сеть в целом.

Каждый путь в свою очередь описывает направление потока только одного конкретного груза. Кроме того, при наличии уже допустимого решения добавление нового пути в рассматриваемое множество путей не ухудшает текущее решение.

В связи с этим в окрестностях VNS предлагается формировать и модифицировать именно множество путей для каждого товара, стремясь приблизиться к оптимальному решению в путевой постановке.

Улучшение текущего решения возможно за счет добавления новых путей, позволяющих более оптимально перераспределить товарные потоки. Поэтому в

рамках каждой окрестности формируется множество новых путей для всех товаров. Полученные пути добавляются в модель, после чего запускается оптимизационный солвер на ограниченное время для перераспределения потоков. В случае улучшения значения целевой функции в текущее решение добавляются все те новые пути, поток по которым оказался ненулевым.

Далее описаны три структуры окрестностей, определяющие способы генерации новых путей на основе текущей модели.

2.3.1. Окрестность кратчайших путей

Одним из распространенных эвристических приемов в задачах потокового распределения является добавление новых кратчайших путей. Однако классические алгоритмы их поиска учитывают только веса ребер, игнорируя дополнительные затраты.

В рассматриваемой задаче кроме затрат на передвижение транспортных средств по дорогам необходимо учитывать стоимость перегруза товара в промежуточных узлах. Для этого на основе исходного полного графа G строится дополнительный G', вес каждого ребра которого соответствует стоимости перевозки одной единицы товара по нему с учетом перегрузки на складах:

$$c'_{vu} = f_v + f_u + \frac{c_{vu}}{C}, \quad \forall (vu) \in E,$$

где c'_{vu} – вес ребра в графе G', соединяющего склады v и u, c_{vu} – стоимость перевозки по исходному ребру из G.

Перед запуском основного алгоритма GVNS для каждого товара предварительно генерируется набор кратчайших путей на основе графа G'. В дальнейшем при формировании множества новых путей на основе заданного решения из этого набора выбираются еще не используемые в нем пути либо случайным образом, либо отдавая предпочтение путям с наименьшими стоимостями.

В качестве параметров данной окрестности выступают количество новых путей, добавляемых для каждого груза, и булевый флаг, определяющий стратегию выбора (случайно или предпочтительно по стоимости).

На этапе VND данные параметры не фиксируются, а перебираются в рамках всех возможных комбинаций: от 1 до 3 новых путей и двух стратегий выбора. В отличие от этого, на этапе Shake-функции всегда добавляется 5 случайно выбранных путей, что способствует более эффективному выходу из локального оптимума.

2.3.2. Окрестность частично загруженных ребер

Структура второй окрестности основывается на предположении, что при формировании новых путей стоит отдавать предпочтение ребрам, на которых уже присутствует частично загруженное транспортное средство. В него можно перегрузить часть товара и освободить другой, тем самым сократив количество используемых транспортных средств.

Для реализации данного принципа на основе исходного графа G строится вспомогательный граф G'', веса ребер в котором учитывают степень загрузки транспортных средств . Предварительно для каждого ребра вычисляется остаток от деления суммарного потока товара по этому ребру на вместимость одного транспортного средства:

$$load_e = \left(\sum_{k \in K} \sum_{p \in \Pi_k : e \in p} x_p^k \right) \bmod C, \ \ \forall e \in E.$$

Ребра, у которых данное значение равно 0 (полностью загруженные), являются наименее приоритетными для выбора, поскольку добавление нового потока по такому ребру потребует запуск дополнительного транспортного средства. В связи с этим вес такого ребра в новом графе устанавливается равным весу в графе G'.

Для остальных ребер вес в графе G'' определяется с учетом как стоимости перегруза на складе, так и свободного места в транспортном средстве:

$$c''_{vu} = f_v + f_u + \frac{load_{vu}}{C} * \left(\frac{c_{vu}}{C}\right)^{\alpha}, \quad \forall (vu) \in E,$$

где c_{vu} – стоимость ребра (vu) в графе G, $\alpha \leq 1$ – коэффициент влияния стоимости ребра. В данном выражении меньшее значение $load_{vu}$ соответствует большему свободному месту и, следовательно, более высокой приоритетности ребра. Чем меньше α, тем меньше на вес ребра влияет стоимость перевозки груза по ребру.

Благодаря такой структуре графа G'' алгоритм поиска кратчайших путей будет преимущественно строить новые пути, проходящие через ребра с частично загруженными транспортными средствами.

Количество новых кратчайших путей, добавляемых в окрестность, определяется динамически в процессе их генерации. Для этого используются две вспомогательные переменные: $count$ для подсчета полностью загруженных путей (содержащих хотя бы одно ребро с $load_e = 0$), и sum для вычисления суммарного свободного места в транспортах на всех сгенерированных путях. При получении очередного пути находится максимальное значение $load_e$ среди его ребер: $load_{max} = \max\limits_{e \in p} load_e$. Если среди ребер есть хотя бы одно с $load_e = 0$, то инкримируется $count$. Иначе к sum прибавляется $1 - load_{max}$. Генерация новых путей заканчивается, когда одна из переменных достигает заданного значения. В качестве параметра для ограничения суммарного свободного места указывается произведение некоторого коэффициента на вместимость одного транспортного средства.

В процессе VND процесс генерация останавливается при наборе максимум 2 путей с полностью заполненными ребрами и при превышении суммарного свободного места вместительности одного транспортного средства. В Shake-функции данные значения увеличиваются до 3 путей и $3*C$ соответственно для рассмотрения большего диапазона возможных путей.

2.3.3. Окрестность обхода перегруженных вершин

Помимо учета загрузки транспортных средств на ребрах, важным фактором является уровень перегруза на складах. Вершины, значение суммарного перегруза на которых равно или близко к их пропускной способности, пользуются высоким спросом и являются «узким» местом. Поэтому целесообразно формировать новые пути в обход таких складов.

На основе текущего решения рассчитывается коэффициент загрузки – отношение суммарного объема перегружаемого товара к максимальной допустимой пропускной способности склада:

$$overload_v = \frac{1}{W_v} * \sum_{\substack{k \in K, \\ v \neq s_k, v \neq t_k}} \sum_{p \in \Pi_k} \sum_{v \in p} x_p^k, \ \forall v \in V$$

Все склады сортируются по убыванию значения $overload_v$. Далее формируется подмножество наиболее перегруженных вершин согласно заданному порогу. Если размер данного множества слишком велик или мал, то оно либо урезается, либо дополняется другим складами до необходимого размера.

Полученный в результате упорядоченный набор $\{v_1, v_2, \dots v_n\}$ используется для генерации согласно следующую алгоритму. Пусть для каждого товара необходимо сгенерировать k новых путей. Тогда для каждого подмножества $\{v_1, v_2, \dots, v_i\}, i = \overline{n-k, n}$ выполняется следующий алгоритм: из графа G' удаляются все вершины из подмножества (за исключением пункта отправления и назначения для конкретного товара), после чего выполняется поиск кратчайшего пути. Количество путей, генерируемых для одной конфигурации исключенных вершин, является параметром окрестности.

Таким образом, формируемые пути представляют собой альтернативные маршруты, избегающие наиболее загруженных складов, что способствует перераспределению потоков на менее загруженные узлы сети.

На этапе VND минимальный размер такого множества составляет пятую часть от общего числа вершин, а максимальный - четверть. Порог перегруженности установлен на уровне 0.75, а для каждой конфигурации исключаемых складов генерируется один новый путь. Для Shake-функции диапазон размера множества расширяется (от четверти до трети вершин), порог перегруженности снижается до 0.5, а количество генерируемых путей на одну конфигурацию увеличивается до трех. Такой подход позволяет существенно разнообразить решение и эффективнее выходит из локальных оптимумов.

2.4. Итоговый вид предлагаемого алгоритма

Предложенные подход полностью следует структуре GVNS, описанной в алгоритме 4. Окрестности перебираются в том порядке, в котором они представлены в данной работе.

Как уже было сказано, для получения решения на расширенном множестве путей используется оптимизационный солвер с ограничением по времени. Продолжительность работы солвера на этапе VND меньше, чем при выполнении Shake-функции, поскольку в рамках VND рассматривается меньшая область решений.

Кроме того, после перераспределения потоков часть путей может стать неиспользуемой (с нулевым потоком). Для предотвращения чрезмерного роста размерности модели периодически производится очистка множества путей с помощью исключения из рассмотрения всех путей с нулевым потоком. Очистка производится каждый раз после выполения Shake-функции, а также после второй и третьей окрестностей на этапе VND в случае улучшения целевой функции.

На работу всего алгоритма в целом устанавливается лимит по времени, однако если после перебора всех окрестностей результат не улучшился, то он останавливается досрочно.

Глава 3. Проведение эксперимента

3.1. Постановка задачи для эксперимента

Одной из задач, поставленных в работе, является оценка эффективности предложенного алгоритма на сетях различной размерности. В связи с тем, что данная постановка задачи достаточно специфична, в открытых источниках отсутствует тестовая база данных с известными оптимальными значениями. Также получение оптимального решения с помощью точных методов требует значительных вычислительных ресурсов и времени. Исходя из этого, оценку качества результата работы алгоритма необходимо проводить на основе более доступных решений задачи. Для этой цели были выбраны:

1. значение целевой функции релаксированной задачи;
2. решение релаксированной задачи с округлением вверх нецелочисленных значениями переменных y_e;
3. решение исходной задачи, полученным с помощью солвера за время, сопоставимое с временем работы тестируемого алгоритма.

В качестве результата эксперимента очевидно ожидать то, что значение релаксирвоанной задачи будет меньше полученного предложенным подходом, а решение с округлением вверх - выше, поскольку он используется в качестве начального приближения алгоритма. Они являются нижней и верхней оценкой соответственно. Решение, полученное солвером за ограниченное время, также должно уступать по качеству, что соответствует основной идее эвристического подхода: получении результата, приближенного к оптимальному, за относительно короткое время.

Особый интерес представляет то, какая именно получится разница между полученными решениями и ее зависимость от размерности задачи. Так как при увеличении размерности существенно возрастает количество возможных путей, то поиск оптимального множества сильно усложняется. В связи с этим ожидается, что при росте размерности сети относительное отклонение решения предлагаемого алгоритма от опорных значений будет увеличиваться.

3.2. Генерация синтетических данных для проведения эксперимента

Поскольку в открытых источниках не удалось найти готовых тестовых наборов данных, подходящих для оценки предложенного алгоритма, возникла необходимость в создании множества синтетических графов и запросов на перевозку товаров, имитирующих реальные логистические сети.

Процесс доставки товара от склада к конечному пользователю чаще всего состоит из нескольких этапов, которые разделяются по масштабу сети складов: от глобального уровня до локального (доставка последней мили). Будут рассматриваться сети верхнего уровня, то есть те графы дорог, которые соединяют региональные сортировочные центры, расположенные в рамках целой страны либо ее части. Подобные крупные складские помещения чаще всего располагаются в центре густозаселенной области вблизи крупных транспортных развязок.

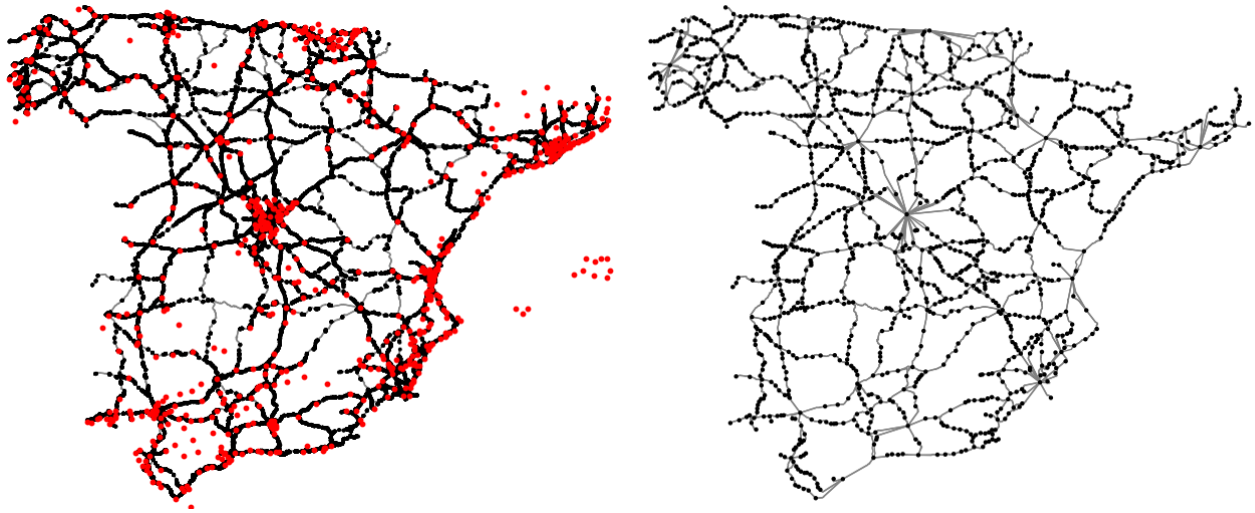
3.2.1. Формирование графов на основе реальных данных

Для генерации правдоподобных данных необходимо произвести анализ реальных сетей. Так как в свободном доступе отсутствует информации о расположении складов существующих служб доставок, то граф формировался на основе данных о расположении городов, их населения и магистралях, их соединяющих. Источником данной информации стал некоммерческий веб-картографический проект OpenStreetMap [9], с помощью которого можно получить данные о транспортных сетях и о населенных пунктах всего мира.

Для рассмотрения были выбраны 9 европейский стран разной площади и структуры, 5 крупных экономических штата США и Европейская часть России.

Исходные данные представляют собой граф, состоящий из большого множества участков магистралей, и набор городов данного региона (рис. 1а). Первым делом необходимо агрегировать множество ребер, соединив в одну вершину близко расположенные концы дорог (на расстоянии нескольких

километров) (рис. 1б). На основе того, что склады чаще расположены рядом с крупными населенными пунктами, производим фильтрацию городов согласно следующему алгоритму: города перебираются в порядке убывания населения и в итоговое множество добавляются те, которые находятся не ближе заданного расстояния к уже выбранным (рис. 1в). Далее для каждого города выбирается одна из ближайших вершин графа дорог с наибольшей степенью (так как выгоднее расположить распределительный центр на пересечении нескольких магистралей) (рис. 1г). Последний этап заключается в упрощении графа путем удаления неиспользуемых вершин и объединения найденных центров ребрами, которые соответствуют кратчайшим путям, не содержащим другие вершины (рис. 1д). Размер каждой вершины равен суммарной численности населения ближайших к ней городов. Учитывание реалистичной формы дорог сильно усложнит процесс генерации графов, в связи с чем в синтетических сетях ребра представляют собой отрезки, напрямую соединяющие вершины и с длиной равной эвклидовому расстоянию между ними (рис. 1е).



а б

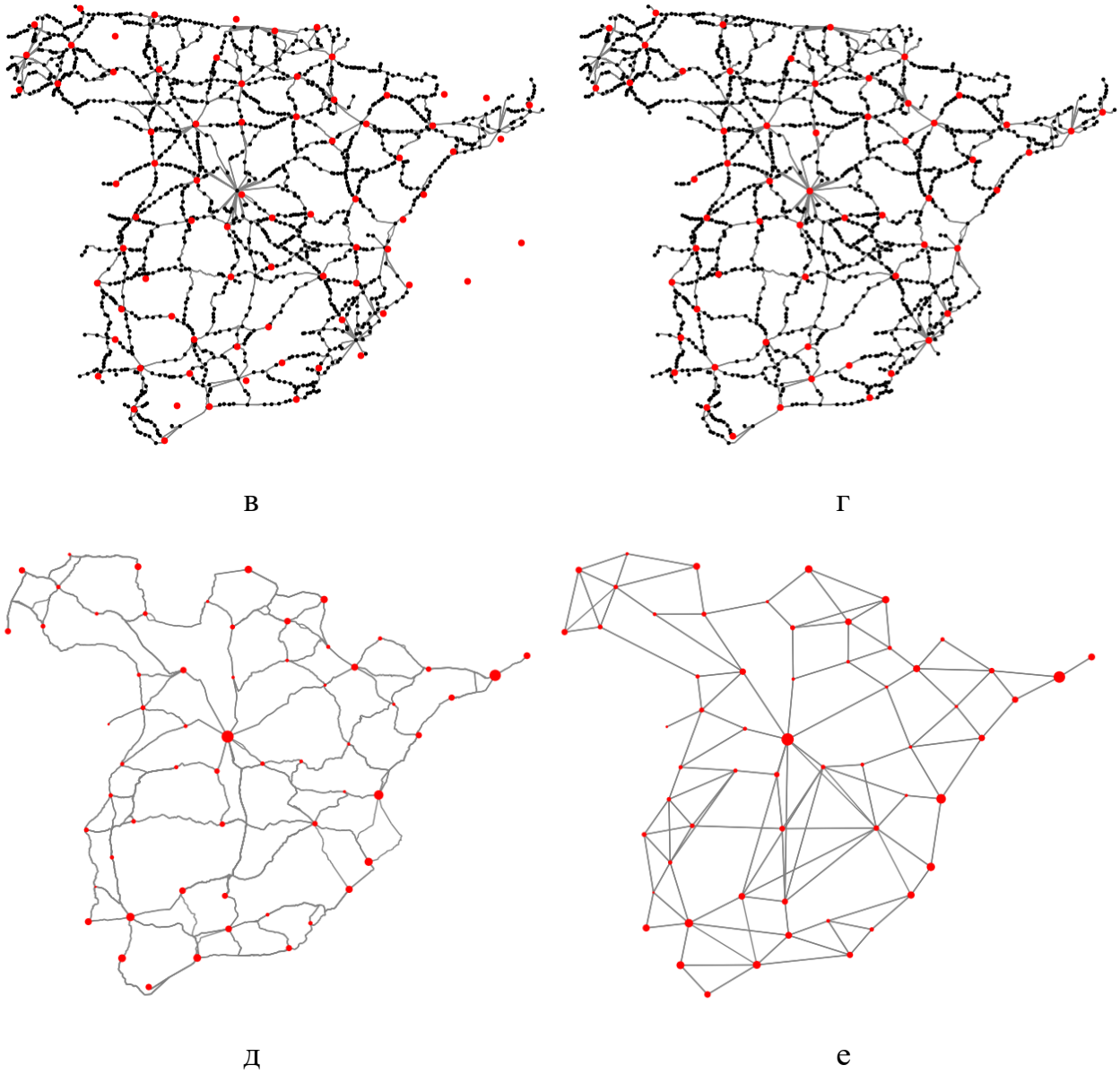


Рисунок 1. Изображение сети дорог Испании. а – сырые данные; б – агрегированы ребра графа; в – агрегированы города; г – выбор вершин графа, соответствующих городам; д – объединение вершин ребрами, соответствующими кратчайшим путям; е – упрощение модели

Полученные графы иллюстрируют возможную схему расположения складов в транспортной сети, основанную на реальных географических данных. Далее полученные графы используются для создания и настройки алгоритма генерации сетей.

3.2.2. Анализ характеристик сформированных графов

В исходных данных каждое ребро описывает участок дороги только в одном направлении, то есть исходный граф является направленным и

необязательно расстояние в одном направление совпадает с обратным. В среднем данные значенич отличаются на 1%. Хоть такая разница и не является критичной, однако данную статистику можно использовать при генерации графа для большей реалистичности.

Как было показано выше, разница между разнонаправленными ребрам, соединяющими одинаковые вершины, не велика, поэтому далее рассматривается неориентированная версия графа. Важной характеристикой взвешенного графа является распределение весов его ребер (рис. 2). Чтобы рассмотреть общую статистику для всех графов, берется отношение длины ребер к расстоянию между самыми далекими вершинами.

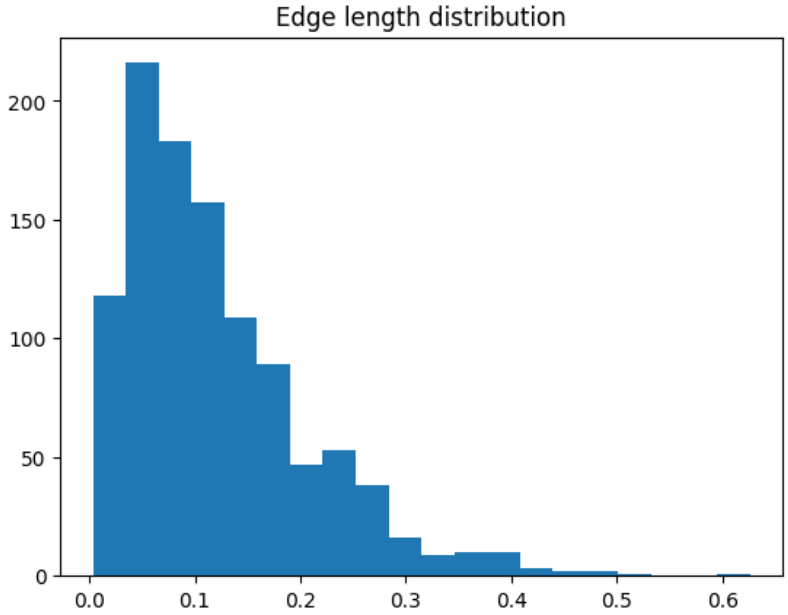


Рисунок 2. Распределение отношений длин ребра к расстоянию между самыми далекими вершинами

Как можно увидеть на гистограмме, в графах превалируют короткие ребра и почти нет длинных ребер, соединяющие далекие вершины.

Дистанцию между двумя вершинами можно задать как эвклидово расстояние, исчисляемое в километрах, и как количество ребер, входящих в кратчайший путь между ними. Наибольшее такое расстояние называется диаметром графа. Значения данной характеристики для рассматриваемых графов представлено ниже:

Таблица 1. Значения диаметров графов

Country	**diameter**	$\|V\|$	**Country**	**diameter**	$\|V\|$	**State**	**diameter**	$\|V\|$
Belgium	5	16	Hungary	7	19	California	12	39
Denmark	5	11	Italy	11	46	Georgia	7	24
Germany	8	49	Netherlands	5	18	Illinois	6	28
Spain	9	59	Sweden	11	45	Ohio	6	22
Europe Russia	16	90	United Kingdom	9	42	Texas	10	58

Полученные диаметры достаточно велики. Исходя из этого можно сделать вывод, что данный граф в основном состоит из коротких ребер, соединяющий близкие вершины, из-за чего пути между далекими вершинами содержат относительно много ребер. Данные графы не обладают характеристиками “small world”, а описывают обратную структуру.

Еще одной важной характеристикой графа является степень вершины. Ниже представлена таблица со средним значением данной метрики для каждого графа:

Таблица 2. Значения степеней графов

Country	d_{mean}	$\|V\|$	**Country**	d_{mean}	$\|V\|$	**State**	d_{mean}	$\|V\|$
Belgium	3.125	16	Hungary	2.737	19	California	2.564	39
Denmark	2.545	11	Italy	3.609	46	Georgia	3.75	24
Germany	5.388	49	Netherlands	3.667	18	Illinois	3.5	28
Spain	4.136	59	Sweden	3.422	45	Ohio	3.727	22
Europe Russia	4.266	90	United Kingdom	4.190	42	Texas	3.0	58

Среднее значение степени равняется 3.575, что характеризует графы как близкие к планарным, так как для них этот показатель не превышает значение 6. В общем все транспортные сети являются почти планарными, то есть графами, которые можно изобразить без пересечения ребер. Большие значения диаметра и полученное распределение длин ребер так же являются следствием планарной формы графа.

3.2.3. Алгоритм генерации данных

Предложенный подход генерации данных для поставленной в рамках этой работы задачи состоит из постепенного построения сети, установки ограничений на максимальный перегруз складов и генерации запросов на перевозку товаров. В качестве входных данных выступает желаемое количество складов k.

1. Генерация городов и их населения

В процессе обработки реальных данных в пункте 3.2.1. работы для формирования складов использовались позиции городов и численность населения в них. Для того, чтобы использовать аналогичный подход, первым делом необходимо сгенерировать населенные пункты с заданной численностью и распределить их по двумерной карте.

Первым этапом является создание набора городов (размером в несколько раз больше желаемого количества складов) с реалистичным распределением численности населения. В науке известен закон Ципфа [10], который является эмпирической закономерностью распределения численности населения в городах. Он заключается в том, что размер населения обратно пропорционален рангу этого населенного пункта:

$$size_i = \frac{P}{rank_i^{\alpha}}, \qquad rank = 1,2,\dots,N,$$

где $rank_i$ – ранг города, P — нормировочная константа, которая равна размеру наибольшего города.

На основе данной закономерности генерируются значения численности населения, к которым добавляется небольшое случайное отклонение для повышения реалистичности распределения.

Коэффициент α в большинстве эмпирических исследований лежит в диапазоне от 1.05 до 1.1. Для оценки соответствия рассматриваемого набора данных этой закономерности было рассчитано значение α для каждого региона.

Между значениями $\log(rank)$ и $\log(size)$ должна быть приблизительно линейная зависимость с наклоном $-\alpha = -1$:

$$\log(size) \sim -\alpha * \log(rank)$$

Результаты значения α на основе данных о городах из пункта 3.2.1 представлены ниже:

Таблица 3. Значение коэффициента α

Country	α	**Country**	α	**State**	α
Belgium	0.764	Hungary	1.005	California	1.010
Denmark	0.913	Italy	0.763	Georgia	0.886
Germany	0.966	Netherlands	0.893	Illinois	0.876
Spain	0.963	Sweden	1.036	Ohio	0.871
Europe Russia	1.338	United Kingdom	1.127	Texas	1.195

Среднее значение коэффициента α на основе наших данных равно 0.974, что соотносится с информацией о линейной зависимости между логарифмами ранга и численности.

Для генерации городов в данной работе параметру α устанавливается случайное число из $\mathcal{N}(0.974, 0.06)$, а в качестве наибольшей численности населения используется значение в диапазоне $(9 * 10^4 * k, 11 * 10^4 * k)$. Число генерируемых городов равно произведению желаемого количества складов на случайное число от 8 до 12.

2. Расположение городов на карте

Для расположения данных городов на двумерной карте заданного размера с помощью средства make_blobs из библиотеки sklearn.datasets генерируются k кластеров со стандартным отклонением в несколько раз меньше размера карты. Данный подход помогает имитировать реальное расположение городов, где вокруг крупного города сосредоточены более маленькие.

Далее из каждого кластера выбираются случайно по одной точке и им назначаются самые крупные города. Все остальные точки распределяются случайным образом. Полученная в результате карта городов соответствует по формату исходным данным, поэтому далее аналогично пункту 3.2.1. происходит группировка вершин в склады. Затем каждому складу присваивается значение, равное суммарной численности городов, ближайших к нему.

Ниже представлен результат, который был получен при генерации 500 городов для получения $k = 50$ складов (размер вершины отражает численность населения) (рис. 3):

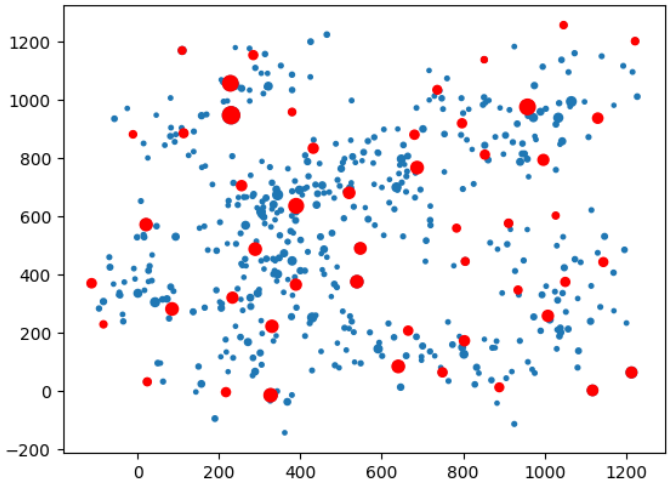


Рисунок 3. Сгенерированная карта городов (синие точки) и складов (красные) для $k = 50$

Минимальное расстояние между складами задается как случайное число от 80 до 100, а размер карты равен произведению полученного значения на $k^{0.8}$. Стандартное отклонение, передаваемое в функцию make_blobs, в 15 раз меньше размера карты.

3. Генерация топологии графа

Самый важный этап генерации – соединить склады ребрами таким образом, чтобы граф был похож на реалистичный. Во втором пункте было показано, что граф должен состоять в основном из коротких ребер, иметь относительно большой диаметр и иметь форму похожую на планарную.

В работе «The spatial structure of networks» [11] авторы анализируют реальные пространственные сети (дороги США, Интернет AS-level, авиаперелёты Delta Airlines). Здесь так же показано, что транспортная сеть близка к планарной, имеет короткие ребра, большой диаметр и низкую степень вершин. Данная информация соотносится с графами, рассматриваемыми в этой работе. Авторы предлагают оптимизационную модель, в которой сеть строится путем минимизации компромисса между общей длиной ребер (стоимость строительства сети) и суммарным «эффективным расстоянием» между

вершинами (удобство пользователей). С точки зрения пользователя сеть будет хорошей, если во время путешествия из одной вершины в другую будет совершено меньше пересадок. Но с экономической точки зрения строить дополнительные длинные ребра затратно.

Положим, что стоимость строительства сети пропорциональна сумме длин ребер d_{ij} (евклидово расстояние в км):

$$sum\ cost = \sum_{edges(i,j)} d_{ij}$$

А «эффективное расстояние» ребра задается как:

$$L_{ij} = \lambda * d_{ij} + (1 - \lambda) * 1, \qquad 0 \leq \lambda \leq 1$$

Параметр λ задает предпочтение пользователя измерять расстояние между вершинами в км (тогда $\lambda = 1$), либо в количествах ребер на пути (тогда $\lambda = 0$). Расстояние между любыми двумя вершинами определяется как сумма «эффективных расстояний» ребер на пути между ними.

Пусть задан набор точек на плоскости, бюджет и параметр λ. Необходимо построить связную сеть в рамках бюджета, минимизируя среднее расстояние между каждой парой вершин. Это стандартная комбинаторная оптимизационная задача, решение которой можно получить, например, методом имитации отжига.

Так как задаваемые бюджет – величина, зависящая от расположения вершин конкретного графа, то удобнее использовать более универсальный параметр, например, какой-то коэффициент. Для получения связного графа в начале алгоритма используется минимальное остовное дерево. Он же используется для определения бюджета, а именно его стоимость, умноженная на заданный коэффициент.

Для определения оптимальных значений данных параметров, далее приведен анализ рассматриваемых реальных графов. Коэффициент бюджета равен частному стоимостей графа и минимального остовного дерева. Однако λ аналитически вычислить невозможно. Поэтому на основе реальных координат вершин генерируются графы с разным параметром λ и сравниваются

характеристики полученного графа (диаметр, средняя длина ребра, и средняя степень вершины) и эталонного. Лучшие λ с точки зрения схожести параметров графов и значения коэффициента бюджета для каждого региона представлены ниже.

Таблица 4. Значения параметров алгоритма

Country	λ	$coeff_{budg}$	**Country**	λ	$coeff_{budg}$	**State**	λ	$coeff_{budg}$
Belgium	0.250	2.026	Hungary	0.225	1.837	California	0.025	1.727
Denmark	0.225	1.640	Italy	0.325	2.441	Georgia	0.075	2.707
Germany	0.350	4.659	Netherlands	0.025	2.629	Illinois	0.300	2.445
Spain	0.500	3.048	Sweden	0.225	2.490	Ohio	0.025	2.550
Europe Russia	0.5	3.244	United Kingdom	0.625	2.925	Texas	0.275	2.032

Как можно увидеть, значения λ в большей степени ниже 0.5, их среднее равно 0.263. Коэффициент бюджета в среднем равен 2.56. Хоть в статье указывается, что для дорожных сетей более характерны значения λ ближе к 1, были получены обратные значения. Это можно объяснить тем, что бюджет является небольшим и его недостаточно для создания длинных ребер между далекими вершинами. Но меньший λ позволяет получить несколько относительно длинных ребер среди большого количества коротких, из-за чего сгенерированный граф и похож на реальную сеть.

Например, для координат, полученных в прошлом пункте, был получен граф, сгенерированный с параметрами, равными полученым средним значениям (рис. 4):

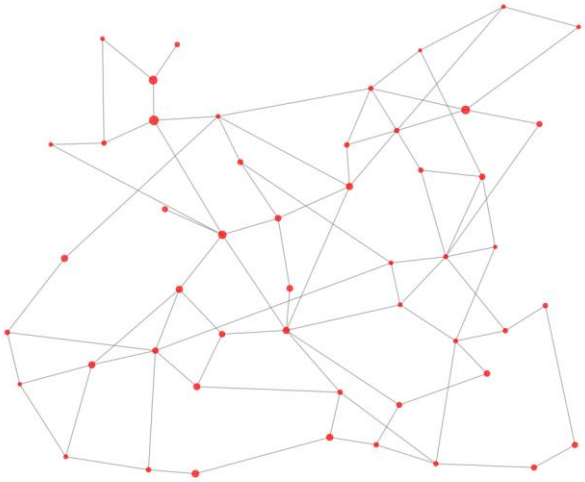


Рисунок 4. Сгенерированный граф, $\lambda = 0.263, coeff_{budg} = 2.56$

Итоговая длина ребра задается как эвклидовою расстояние между его вершинами с добавление небольшого случайного отклонения. Как было сказано на этапе анализа реальных сетей, оригинальные графы являются направленными и длины ребер с одинаковыми концами отличаются в среднем на 1%. Поэтому для каждого ребра в графе создается второе в обратном направление и длиной, отличающимся от него максимум на сотую долю.

Для задачи необходимо задать стоимость проезда одного транспортного средства по ребру, что в общем прямо пропорционально длине дороги. Поэтому после генераций ребер им устанавливается стоимость использования по следующей формуле:

$$c_e = cost_{km} * dist_e,$$

где $cost_{km}$ – стоимость проезда одного километра.

В рамках данной работы генерация топологии графы производилась с параметрами λ из $\mathcal{N}(0.263, 0.15)$ и $coeff_{budg}$ из $\mathcal{N}(2.56, 0.6)$. Стоимость проезда транспортным средством одного километра равна 5.

4. Генерация стоимости и максимального количества перегруза товара на складе

Максимальная пропускная способность склада должна соответствовать потенциальному спросу и потоку, который может проходить через вершину.

Первая характеристика может быть отражена численностью населения, обслуживаемого складом. А возможный поток, согласно некоторым работам [12], [13], имеет высокую корреляцию с betweenness centrality вершины (отношение количества кратчайших путей, проходящих через вершину, к общему количеству кратчайших путей). Поэтому складу назначается следующее значение максимального перегруза:

$$max\ overload_i = l * population_i^{\gamma}\left(1 + \alpha\widetilde{b_i}\right),$$

где $popualtion$ – население в данной вершине; $\widetilde{b_i}$ – нормализованное betweenness centrality вершины i; γ, α – коэффициент влияния численности населения и betweenness; l – коэффициент пропорциональности.

Стоимость перегруза на складах обратно пропорциональна вместимости склада. Это отражает эффект масштаба и более высокую эффективность крупных логистических центров.

$$cost_i = \frac{k}{max\ overload_i^{\delta}},$$

где k – коэффициент пропорциональности, δ – коэффициент влияния вместимости склада.

Генерация пропускных способностей складов происходила со следующими параметрами: $l = 1, \gamma = 0.5, \alpha = 1.2$. Также задано минимальное значение перегруза $min\ capacity = 100$. Так как при разных порядках population значения $max\ overload$ могут сильно разниться, то лучше адаптировать значения максимального перегруза таким образом, чтобы хотя бы 10% из них были меньше $min\ capacity$.

Для получения стоимости перегруза для каждого склада использовались следующие значения переменных: $k = 500, \delta = 1$.

5. Генерация запросов на перевозку товаров

Ключевым параметром задачи распределения потока является матрица корреспонденций (Origin-Destination matrix, OD-матрица), элементами которой

T_{ij}^{k} выступает объем k-ого товара, который необходимо перевезти из вершины i в j.

Для генерации OD-матрицы часто используется гравитационная модель, которая основана на аналогии с законом всемирного тяготения, где интенсивность взаимодействия между двумя точками пропорциональна произведению их «масс» и обратно пропорциональна расстоянию между ними [14]. В общем виде doubly constrained gravity model (двойно-сбалансированная гравитационная модель) записывается следующим образом:

$$T_{ij} = A_i * O_i * B_j * D_j * f(dist_{ij}),$$

где O_i, D_j – суммарное количество груза, отправляемого из вершины i и привозимого в j соответственно; $f(dist_{ij})$ – функция сопротивления, зависящая от расстояния между вершинами; A_i, B_j – балансирующие факторы, обеспечивающие выполнение условий: $\sum_j T_{ij} = O_i, \sum_i T_{ij} = D_i$.

Значения O_i и D_j определяются пропорционально населению вершины с добавлением небольшого случайного возмущения для повышения реалистичности, а в качестве $dist_{ij}$ используется кратчайшее расстояние между вершинами i и j. Сама функция сопротивления часто имеет вид $f(dist_{ij}) = dist_{ij}^{-\beta}$. Балансирующие множители A_i и B_j вычислялись итеративным методом до достижения заданной точности.

В процессе генерации данных для тестирования в качестве множеств O и D использовалась численность населения, обслуживаемого складом, умноженного на небольшое отклонение. Однако, чтобы адаптировать суммарный поток под пропускную способность складов, он скалируется так, чтобы был в несколько раз больше суммы максимального перегруза. Параметр β, обозначающего степень влияния расстояния между пунктом отправления и назначения на результирующий поток, равен 1.

Результатом генерации является полностью заполненная матрица корреспонденций, однако не всегда существует запрос из каждой вершины в

каждую. Поэтому часть (а именно 25%) запросов отбрасывается с вероятностью, пропорциональной их объему.

3.2.4. Составление набора тестовых данных

В сетях, сформированных на основе реальной информации, количество складов варьируется от 11 до 90. Поэтому для тестирования алгоритма было сгенерировано 10 набора данных для 10 складов, по 7 сетей для 20 и 30 складов, 5 для 50, 3 для 100 и 1 для 150.

Кроме полностью синтетических данных было проведено исследование и для транспортных сетей рассмотренных регионов. Так как в данном случае уже есть граф с заданными расположением вершин и населением, то остается сгенерировать стоимости ребер, пропускную способность складов и их цену перегруза, а также множество запросов на перевозку товаров.

Основные характеристики сгенерированных данных описаны в Приложении 1 (таблицы 5 и 6).

3.3. Критерии оценки

Для каждого экземпляра сгенерированных данных были получены следующие значения:

- GVNS – результат решения задачи предложенным алгоритмом GVNS;
- LP-relaxation – оптимальное значение релаксированной задачи;
- Rounded LP – решение релаксированной задачи с округление вверх нецелочисленных значения переменных y_e;
- Time-limited MILP – решение исходной задачи оптимизационным солвером за время, сопоставимое с временем работы алгоритмом GVNS.

В качестве метрики для дальнейшего анализа результата удобно использовать отношение каждого из полученных решений к результату предложенного алгоритма:

$$Ratio_{method} = \frac{Result_{method}}{Result_{GVNS}}.$$

3.4. Аппаратные характеристики

Рассматриваемый эксперимент проводился на компьютере со следующей конфигурацией:

- процессор AMD Ryzen 5 5600H with Radeon Graphics 3.30 GHz;
- оперативная память 16 Гб;
- 64-разрядная операционная система, процессор x64.

3.5. Результаты эксперимента

Для оценки эффективности предложенного алгоритма GVNS каждый тестовый экземпляр решался с ограничением по времени, равным удвоенному количеству складов в минутах. График, иллюстрирующий изменение значения целевой функции после очередной окрестности в процессе работы алгоритма для сети California_USA, представлен на рисунке 5:

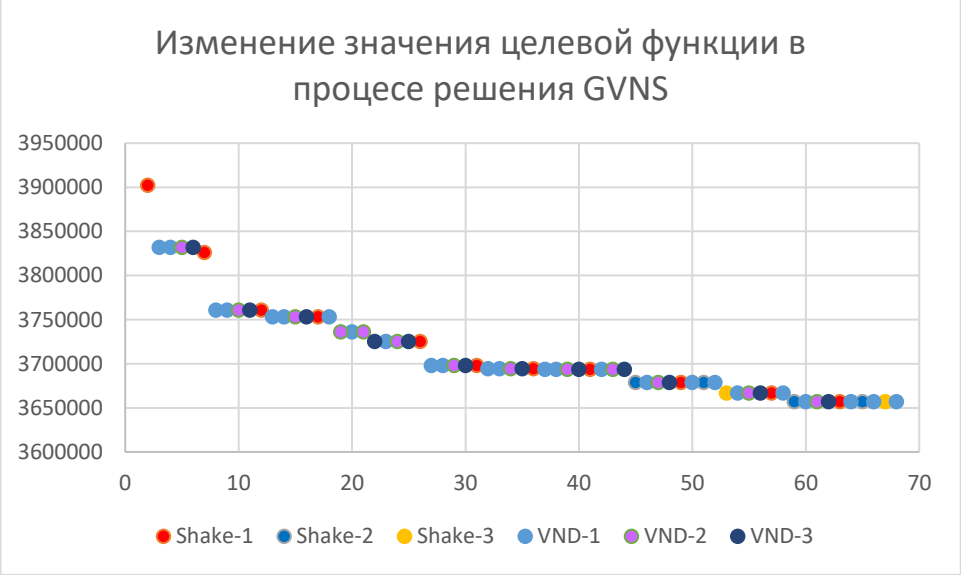


Рисунок 5. Изменение значения целевой функции после очередной окрестности в процессе решения задачи для сети California_USA

Для графов, размерностью 100 и 150, а также для сети Европейской части России не удалось получить решение Time-limited MILP из-за аппаратных ограничений.

Результаты экспериментов представлены на рисунках 6-13. Графики 6-8 и 10-12 отображают отношение целевой функции каждого метода к результату GVNS для синтетических и реальных данных соответственно. Рисунки 9 и 13 представляют сводные графики всех методов. Данные, на основе которых построены графики, представлены в Приложении 2 (таблицы 7 и 8).

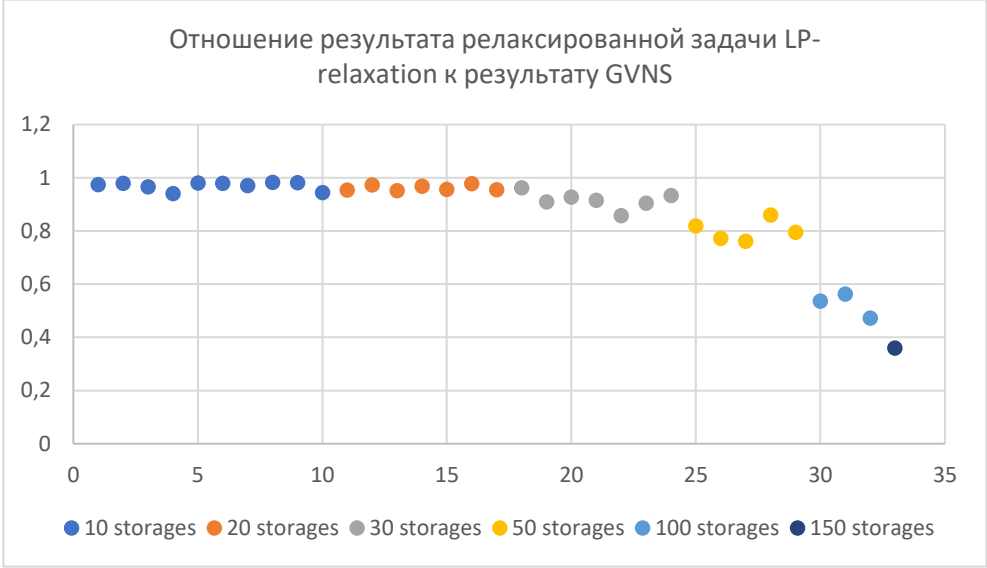


Рисунок 6. Отношение результата релаксированной задачи к результату GVNS для синтетических данных

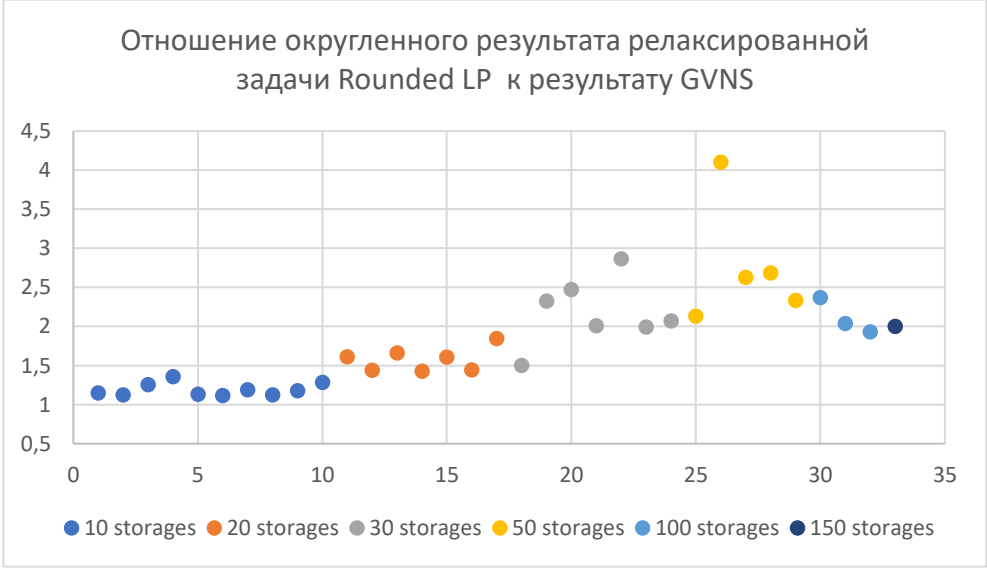


Рисунок 7. Отношение округленного результата релаксированной задачи к результату GVNS для синтетических данных

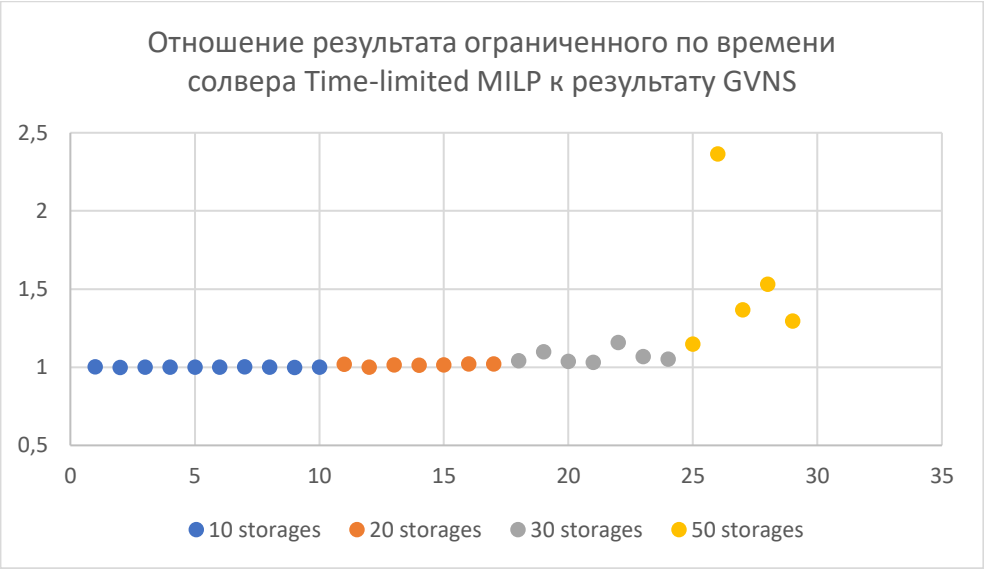


Рисунок 8. Отношение результата ограниченного по времени солвера к результату GVNS для синтетических данных

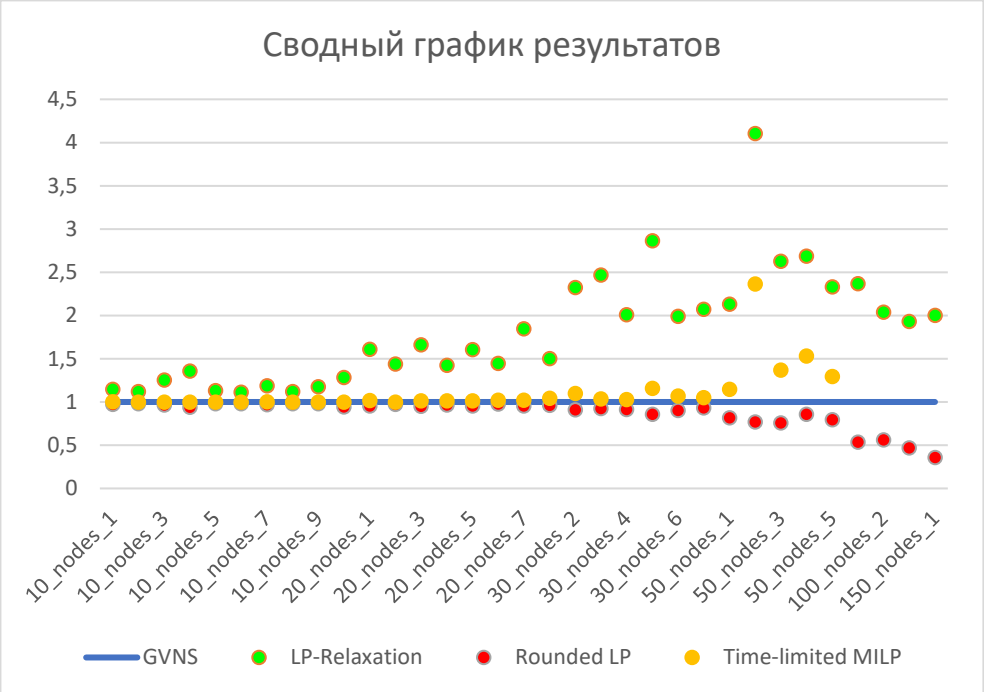


Рисунок 9. Сводный график результатов для синтетических данных

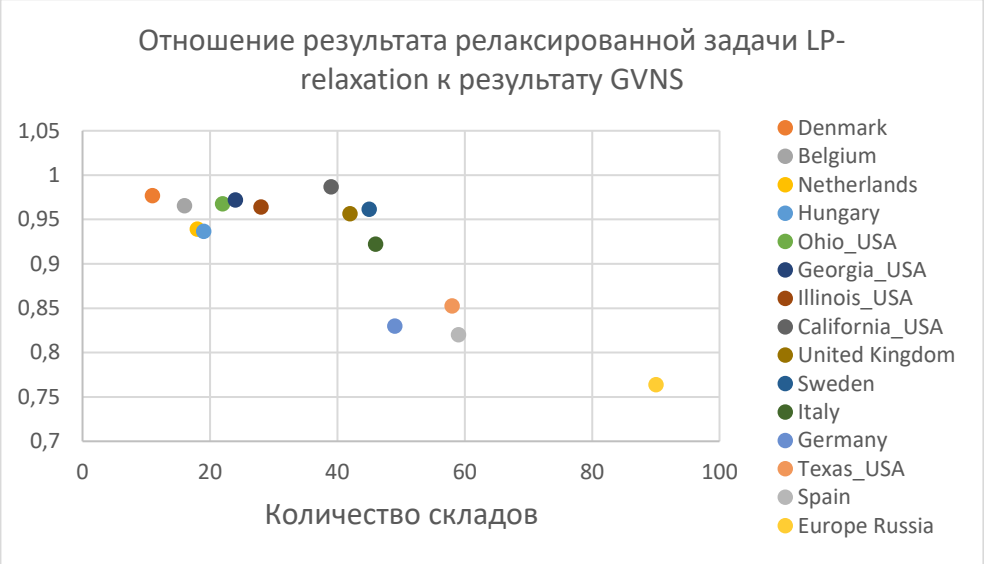


Рисунок 10. Отношение результата релаксированной задачи к результату GVNS для реальных сетей

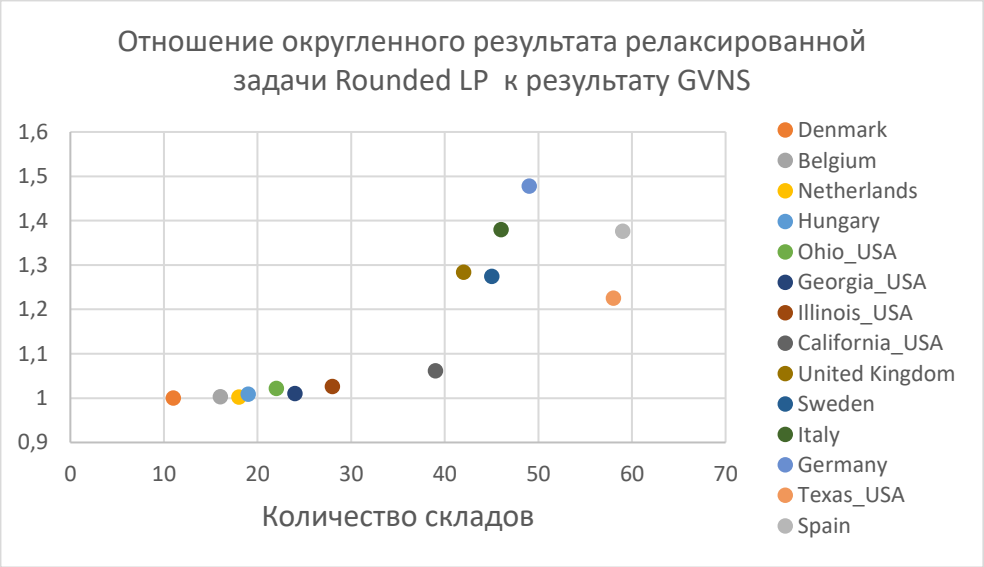


Рисунок 11. Отношение округленного результата релаксированной задачи к результату GVNS для реальных сетей

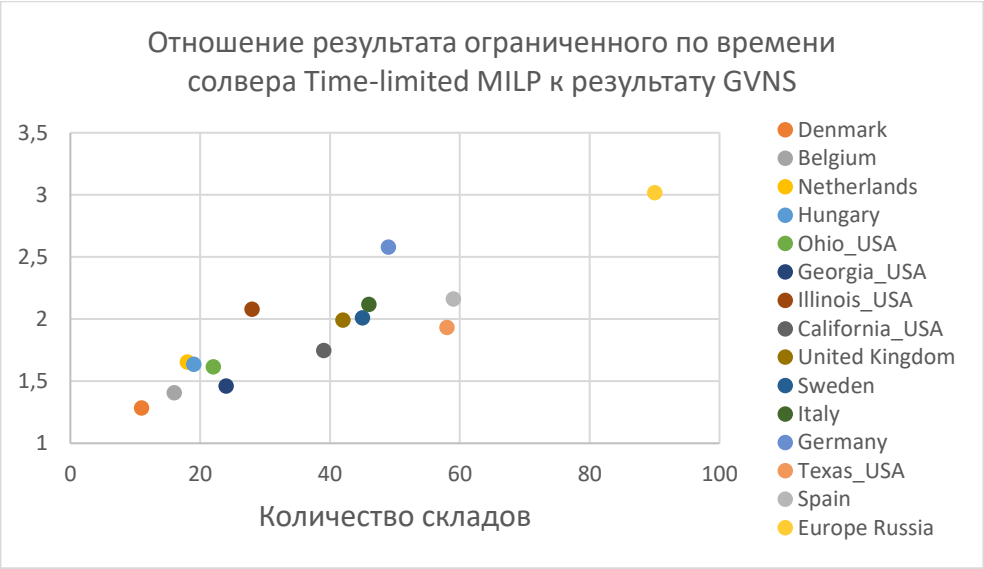


Рисунок 12. Отношение результата ограниченного по времени солвера к результату GVNS для реальных сетей

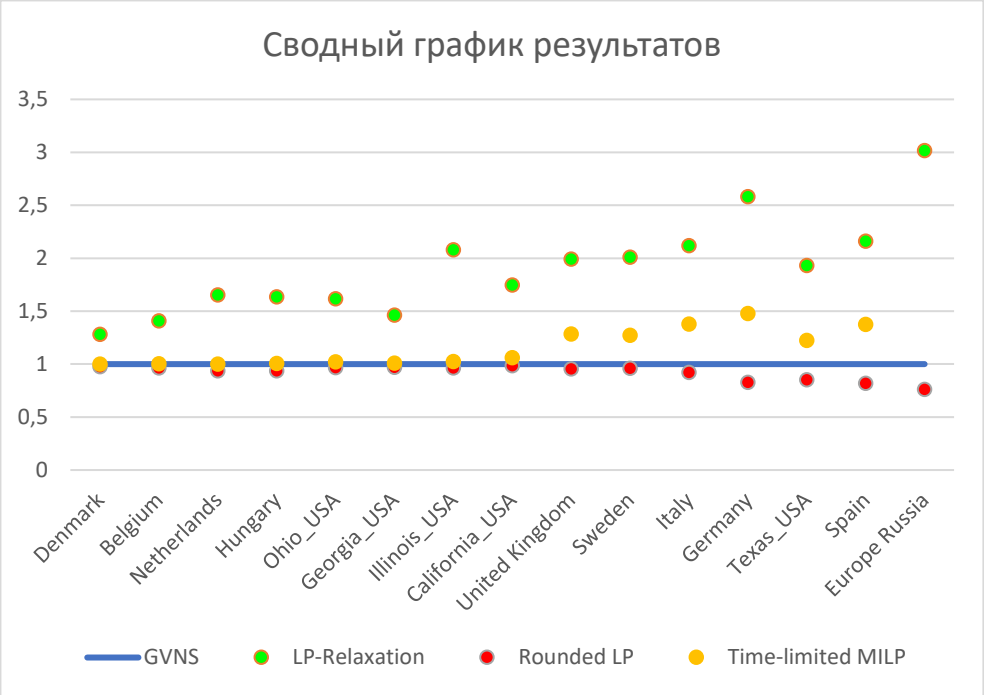


Рисунок 13. Сводный график результатов для реальных сетей

Экспериментальные результаты полностью подтвердили предположения, выдвинутые на этапе постановки задачи.

Значение релаксированной задачи (нижняя оценка) для малых размерностей находится близко к результату GVNS и постепенно удаляется с ростом количества складов. Округленное релаксированное решение, использованное в качестве начального приближения (верхняя оценка), очевидно

показывает худшие результаты, а разрыв между значениями также увеличивается с ростом размерности задачи. При этом решения, полученные алгоритмом GVNS, значительно ближе к нижней границе (релаксации), чем к верхней (округленному решению релаксации), что говорит о высокой эффективности предложенного подхода.

Результаты Time-limited MILP примерно повторяют поведение решения Rounded LP, однако располагаются немного ближе к GVNS. Причем разница между релаксированным решением и результатами предложенного подхода все равно меньше разницы с решением солвера, что демонстрирует значительно лучшее качество решения за сопоставимое время.

Графики, построенные как для полностью синтетических данных, так и для реальных сетей регионов, имеют примерно одинаковые тенденции изменения относительных показателей при увеличении размерности задачи. Это подтверждает адекватность разработанного генератора синтетических данных и устойчивость предложенного алгоритма GVNS.

Заключение

В рамках данной работы был разработан алгоритм построения оптимальной логистической транспортной сети с учетом нелинейной стоимости ребер и ограничением на максимальный объем перегружаемого товара в вершинах.

В ходе исследования была построена математическая модель задачи многопродуктового потока минимальной стоимости в потоковой и путевой постановке с учетом заданных ограничений. В качестве основного метода ее решения был выбран General Variable Neighborhood Search, для которого был разработан набор из трех специализированных окрестностей, позволяющих эффективно исследовать пространство решений:

- окрестность кратчайших путей с учетом стоимости перегруза товара на складах;
- окрестность использования незаполненных транспортных средств;
- окрестность обхода перегруженных ребер.

Для проведения вычисления эксперимента для определения эффективности предлагаемого метода, был разработан подход для генерации синтетических логистических сетей, основанных на анализе реальных транспортных графов.

В результате проведенных экспериментов на синтетических и реальных данных (сети европейский стран, европейской части России и отдельных штатов США) была показана эффективность предложенного GVNS алгоритма. Сравнение с результатами релаксированной задачи (нижняя оценка), округленного релаксированного решения (верхняя оценка) и решением точного оптимизационного солвера за сопоставимое время показало, что разработанный подход стабильно находит качественные решения, близкие к нижней границе (однако, удаляющиеся от нее с ростом размерности задачи).

Данную работу можно развивать в сторону модернизации описанных окрестностей и составления новых, также учитывающих специфику задачи. Кроме того, предложенный алгоритма может быть использован на практике при распределении потоков движения товара в реальных логистических сетях.

Список использованной литературы

1. Barnhart C., Johnson E. L., Nemhauser G. L., Savelsbergh M. W. P., Vance P. H. Branch-and-Price: Column Generation for Solving Huge Integer Programs // Operations Research. – 1998. – Vol. 46, No. 3. – P. 316–329.

2. Crainic T. G., Gendreau M., Farvolden J. M. Metaheuristics for Multi-Commodity Flow Problems // Parallel Computing. – 2015. – Vol. 41. – P. 1–15.

3. Masri H., Krichen S., Guitouni A. A multi-start variable neighborhood search for solving the single path multicommodity flow problem // Applied Mathematics and Computation. – 2015. – Vol. 251. – P. 132–142.

4. Amous M., Toumi S., Jarboui B., Eddaly M. A variable neighborhood search algorithm for the capacitated vehicle routing problem // Computers & Operations Research. – 2017. – Vol. 88. – P. 234–248.

5. Hansen P., Mladenović N. Variable Neighborhood Search // Handbook of Metaheuristics / ed. by M. Gendreau, J.-Y. Potvin. – 3rd ed. – Springer, 2019. – P. 57–97.

6. Hansen P., Mladenovic N., Todosijevic R., Hanafi S. Variable neighborhood search: Basics and variants // EURO Journal on Computational Optimization. – 2017. – Vol. 5, No. 3. – P. 423–454.

7. Karakostas P., Sifaleras A., Georgiadis M. C. A general variable neighborhood search-based solution approach for the location-inventory routing problem with distribution outsourcing // Computers & Chemical Engineering. – 2019. – Vol. 126. – P. 263–279.

8. Mjirda A., Jarboui B., Mladenović N., Wilbaut C., Hanafi S. A general variable neighbourhood search for the multi-product inventory routing problem // International Journal of Production Research. – 2014.

9. OpenStreetMap [Электронный ресурс]. – Режим доступа: https://www.openstreetmap.org (дата обращения: 11.04.2026).

10. Gabaix X. Zipf’s Law for Cities: An Explanation // Quarterly Journal of Economics. – 1999. – Vol. 114, No. 3. – P. 739–767.

11. Gastner M. T., Newman M. E. J. The spatial structure of networks // The European Physical Journal B. – 2006. – Vol. 49, No. 2. – P. 247–252.

12. Barthélemy M. Spatial networks // Physics Reports. – 2011. – Vol. 499, No. 1–3. – P. 1–101.

13. Kirkley A., Barbosa H., Barthelemy M., Ghoshal G. From networks to optimal higher-order models of complex systems // Nature Physics. – 2018. – Vol. 14. – P. 250–258.

14. Wilson A. G. A statistical theory of spatial distribution models // Transportation Research. – 1967. – Vol. 1, No. 3. – P. 253–269.

Приложение 1. Характеристики сгенерированных данных

Обозначения в таблицах:

- $|V|$ - количество складов
- deg_{mean} – средняя степень вершин
- $diam$ – диаметр графа
- $dist_{mean}$ – средняя длина ребер в графе
- W_{mean}, W_{sum} – среднее и суммарное значение максимального перегруза складов
- f_{mean} – средняя стоимость перегруза 1 единицы товара
- $|K|$ - количество товаров
- d_{mean}, d_{sum} – средний и суммарный объем товара

Таблица 5. Основные характеристики реальных графов со сгенерированными параметрами вершин и запросами на перевозку товаров

Name	$\|V\|$	deg_{mean}	$diam$	$dist_{mean}$	W_{mean}	W_{sum}	f_{mean}	$\|K\|$	d_{mean}	d_{sum}
Belgium	16	3,13	5	76,31	356,4	5703	2,11	180	87,3	15717,10
Denmark	11	2,55	5	130,39	299,5	3294	2,35	82	101,5	8322,94
Germany	49	5,39	8	207,05	291,3	14276	2,54	1764	24,8	43705,17
Hungary	19	2,74	7	103,32	240,3	4566	3,26	256	80,3	20565,71
Italy	46	3,61	11	134,76	310,6	14288	2,40	1552	39,9	61953,25
Netherlands	18	3,67	5	97,28	260,8	4694	2,73	229	50,7	11605,13
Spain	59	4,14	9	151,49	335,4	19789	2,38	2566	30,3	77666,48
Sweden	45	3,42	11	225,12	475,0	21373	1,83	1485	57,7	85687,87
United Kingdom	42	4,19	9	171,24	574,0	24109	1,75	1291	59,7	77118,87
California_USA	39	2,56	12	138,44	937,0	36544	1,48	1111	102,9	114358,90
Georgia_USA	24	3,75	7	112,44	321,7	7720	2,57	414	84,1	34831,85
Illinois_USA	28	3,50	6	109,62	367,6	10292	2,53	567	42,9	24337,34
Ohio_USA	22	3,73	6	106,89	335,0	7371	2,58	346	66,5	22994,04
Texas_USA	58	3,00	10	138,58	478,7	27767	2,31	2479	44,6	110540,02
Europe Russia	90	4,27	16	394,10	472,1	42492	2,03	6007	22,1	132569,97

Таблица 6. Основные характеристики полностью сгенерированных данных

Name	$\|V\|$	deg_{mean}	$diam$	$dist_{mean}$	W_{mean}	W_{sum}	f_{mean}	$\|K\|$	d_{mean}	d_{sum}
10_nodes_1	10	3,20	4	191,08	256,80	2568	2,57	67	166,63	11164,34
10_nodes_2	10	3,60	3	185,95	296,30	2963	2,70	67	192,24	12880,06
10_nodes_3	10	3,60	3	243,61	221,10	2211	2,95	67	119,65	8016,44
10_nodes_4	10	3,80	3	204,06	191,20	1912	3,09	67	69,94	4686,17
10_nodes_5	10	3,80	3	169,86	205,00	2050	3,13	67	191,29	12816,34
10_nodes_6	10	3,60	3	157,64	237,40	2374	2,33	67	146,60	9822,44
10_nodes_7	10	4,40	3	214,67	183,10	1831	3,09	67	115,79	7757,74
10_nodes_8	10	4,20	3	179,68	542,00	5420	1,73	67	286,09	19167,99
10_nodes_9	10	3,20	4	175,36	314,10	3141	2,03	67	168,36	11280,18
10_nodes_10	10	3,40	4	173,23	212,90	2129	2,72	67	76,92	5153,60
20_nodes_1	20	4,30	5	261,35	172,00	3440	3,36	285	52,41	14936,69
20_nodes_2	20	3,70	6	227,20	262,85	5257	2,37	285	69,37	19770,31
20_nodes_3	20	4,20	5	287,07	192,70	3854	3,00	285	44,44	12665,44
20_nodes_4	20	4,60	5	290,36	206,75	4135	2,96	285	56,21	16019,86
20_nodes_5	20	4,20	5	268,05	218,75	4375	2,75	285	51,01	14537,91
20_nodes_6	20	3,20	7	178,16	314,95	6299	2,19	285	79,96	22787,59
20_nodes_7	20	4,20	4	318,91	244,85	4897	2,62	285	43,54	12408,96
30_nodes_1	30	2,33	11	218,96	196,50	5895	3,02	652	71,90	46879,17
30_nodes_2	30	2,53	10	235,36	193,00	5790	2,97	652	28,12	18336,11
30_nodes_3	30	3,53	7	237,51	228,47	6854	2,71	652	26,34	17172,90
30_nodes_4	30	3,60	8	280,05	189,67	5690	3,11	652	35,37	23060,02
30_nodes_5	30	3,40	8	262,46	168,50	5055	3,30	652	20,82	13575,79
30_nodes_6	30	4,00	5	291,22	197,37	5921	3,13	652	33,39	21768,97
30_nodes_7	30	3,33	6	306,72	220,07	6602	2,77	652	36,96	24098,18
50_nodes_1	50	3,76	7	359,73	180,24	9012	3,32	1837	30,12	55335,95
50_nodes_2	50	2,56	12	238,37	182,32	9116	3,20	1837	13,09	24040,91
50_nodes_3	50	3,04	10	289,67	173,92	8696	3,30	1837	18,85	34633,34
50_nodes_4	50	2,96	10	318,21	205,04	10252	3,06	1837	25,40	46667,66
50_nodes_5	50	3,60	8	318,08	172,54	8627	3,22	1837	22,17	40729,79
100_nodes_1	100	2,76	13	473,42	177,52	17752	3,34	7425	15,67	116313,31
100_nodes_2	100	3,02	13	411,62	192,37	19237	3,08	7425	19,41	144097,88
100_nodes_3	100	3,38	10	557,79	166,05	16605	3,38	7425	16,51	122554,12
150_nodes_1	150	4,28	17	402,79	182,66	27399	3,17	16762	13,62	228333,69

Приложение 2. Результаты эксперимента

Таблица 7. Результаты эксперимента для полностью сгенерированных данных

Name	VNS	LP-relaxation	Rounded LP	Time-limited MILP	Ratio LP-relaxation	Ratio Rounded LP	Ratio Time-limited MILP	Time, сек
10_nodes_1	291527,79	284116,81	334966,61	292413,27	0,97458	1,14900	1,00304	212,21
10_nodes_2	363938,16	356638,95	408837,40	364052,57	0,97994	1,12337	1,00031	356,78
10_nodes_3	245043,86	236744,29	307417,12	245492,75	0,96613	1,25454	1,00183	497,84
10_nodes_4	142248,97	133790,21	193502,88	142505,64	0,94054	1,36031	1,00180	898,66
10_nodes_5	318337,49	312321,37	361027,56	318704,85	0,98110	1,13410	1,00115	184,61
10_nodes_6	282170,70	276498,94	314860,57	282358,10	0,97990	1,11585	1,00066	278,58
10_nodes_7	252108,06	244853,08	299974,16	252744,25	0,97122	1,18986	1,00252	279,23
10_nodes_8	383679,03	377229,15	430977,98	384381,39	0,98319	1,12328	1,00183	135,62
10_nodes_9	315645,35	309966,59	371664,85	315680,08	0,98201	1,17748	1,00011	250,37
10_nodes_10	135640,90	128113,51	174518,98	135770,11	0,94450	1,28663	1,00095	1151,14
20_nodes_1	754557,98	720306,50	1215420,63	768885,52	0,95461	1,61077	1,01899	1461,84
20_nodes_2	887169,06	863762,55	1279015,92	887903,44	0,97362	1,44168	1,00083	1350,52
20_nodes_3	820728,21	780815,48	1363579,23	833780,84	0,95137	1,66143	1,01590	1405,16
20_nodes_4	1024673,41	991993,11	1462323,30	1039735,65	0,96811	1,42711	1,01470	1422,32
20_nodes_5	741963,04	709602,39	1192743,39	753510,22	0,95639	1,60755	1,01556	1090,54
20_nodes_6	876880,56	858338,38	1268649,62	895564,32	0,97885	1,44678	1,02131	855,80
20_nodes_7	735127,99	702564,51	1358555,70	750499,01	0,95570	1,84805	1,02091	1450,65
30_nodes_1	3491402,81	3361357,29	5244986,13	3640770,93	0,96275	1,50226	1,04278	1992,65
30_nodes_2	1718341,72	1563952,99	3994172,20	1889876,21	0,91015	2,32443	1,09983	1231,29
30_nodes_3	1223003,93	1134241,06	3022238,83	1269306,34	0,92742	2,47116	1,03786	2221,51
30_nodes_4	1683406,62	1540868,08	3383696,58	1736307,34	0,91533	2,01003	1,03142	1922,85
30_nodes_5	990993,45	849987,46	2840874,11	1148532,44	0,85771	2,86669	1,15897	1425,33
30_nodes_6	1451379,73	1312846,48	2893059,52	1550889,53	0,90455	1,99332	1,06856	1317,81
30_nodes_7	1789354,43	1671057,77	3708648,52	1882110,85	0,93389	2,07262	1,05184	1825,20
50_nodes_1	6377220,37	5227184,07	13590690,82	7332466,64	0,81966	2,13113	1,14979	4543,87
50_nodes_2	2886888,43	2229228,62	11848502,46	6831264,51	0,77219	4,10425	2,36631	6031,64
50_nodes_3	4409250,82	3355974,21	11586104,89	6035669,52	0,76112	2,62768	1,36887	4955,67
50_nodes_4	5535896,61	4764025,94	14873445,38	8481565,86	0,86057	2,68673	1,53210	4961,25
50_nodes_5	5144186,29	4088894,12	12002526,21	6669219,68	0,79486	2,33322	1,29646	3633,80
100_nodes_1	50902441,46	27297743,89	120630014,43	-	0,53628	2,36983	-	12033,47
100_nodes_2	44393215,92	24997565,43	90500838,57	-	0,56309	2,03862	-	10792,71
100_nodes_3	46653467,08	22040640,78	90179757,39	-	0,47243	1,93297	-	12148,04
150_nodes_1	141570015,25	50867357,47	283415344,92	-	0,35931	2,00194	-	18066,86

Таблица 8. Результаты эксперимента для данных по регионам

Name	$\|V\|$	VNS	LP-relaxation	Rounded LP	Time-limited MILP	Ratio LP-relaxation	Ratio Rounded LP	Ratio Time-limited MILP	Time, сек
Belgium	11	259442,83	253350,10	332714,07	259545,77	0,97652	1,28242	1,00040	609,31
Denmark	16	209595,78	202380,83	295235,34	210272,47	0,96558	1,40859	1,00323	1041,02
Germany	18	201039,27	188844,30	332645,37	201551,89	0,93934	1,65463	1,00255	1026,49
Hungary	19	426875,39	399910,38	698502,69	430631,24	0,93683	1,63632	1,00880	488,46
Italy	22	493799,41	477969,36	798854,18	504552,59	0,96794	1,61777	1,02178	2628,93
Netherlands	24	782384,22	760576,94	1144278,63	790469,18	0,97213	1,46255	1,01033	1791,58
Spain	28	669160,29	645201,56	1392708,96	686836,92	0,96420	2,08128	1,02642	3382,14
Sweden	39	3657037,45	3609121,39	6396638,62	3882643,30	0,98690	1,74913	1,06169	4146,69
United Kingdom	42	2967215,10	2838897,01	5913358,54	3810490,26	0,95675	1,99290	1,28420	2670,61
California_ USA	45	5315899,89	5113025,76	10691648,24	6774765,32	0,96184	2,01126	1,27443	4987,15
Georgia_ USA	46	3147477,05	2902635,71	6669578,37	4343159,02	0,92221	2,11902	1,37989	2770,28
Illinois_ USA	49	2293350,84	1903865,54	5921272,23	3390095,32	0,83017	2,58193	1,47823	3308,13
Ohio_USA	58	6183754,11	5272734,70	11956660,15	7578508,36	0,85268	1,93356	1,22555	3021,88
Texas_USA	59	4583231,15	3759352,38	9911575,03	6307904,85	0,82024	2,16257	1,37630	4086,86
Europe Russia	90	21975316,76	16786520,81	66338414,96	-	0,76388	3,01877	-	11119,15